多模态大模型系列报告

让大象起舞：多模态大模型的高效化与端侧部署

从量化、剪枝到投机解码与视觉 token 压缩——如何让数百亿参数的”大象”在手机与单卡 GPU 上轻盈起舞

多模态大模型系列报告

2026-09-29

本报告为系列技术综述，内容基于公开论文整理，仅供学习研究使用。

目录

1 引言：效率在多模态版图中的位置 4

2 基础知识 5

2.1 推理的两个阶段：prefill 与 decode 5
2.2 KV cache 显存公式：一行推导读懂所有 KV 优化 5
2.3 算术强度与 roofline：为什么” 小批量 decode” 天然慢 6
2.4 量化基础：从均匀量化到” 难度转移” 6
2.5 投机解码的数学保证：为什么” 起草-验证” 是无损的 7
2.6 算子层的地基：FlashAttention 与” 显存读写比 FLOPs 更贵” 7
2.7 剪枝与蒸馏：两种” 变小” 的哲学 7
2.8 多模态的特殊性：视觉 token 的数量级 8
2.9 端侧硬件约束：为什么是 int4 而不是别的 8

3 经典方法谱系 8

3.1 关键论文一览表 9

4 代表作深度剖析 11

4.1 vLLM 与 PagedAttention：把操作系统请进 GPU（SOSP 2023） 11
4.2 GPTQ 与 AWQ：量化双雄的两种世界观（ICLR 2023 / MLSys 2024） 12
4.3 FastV：发现” 图像在第二层之后只值一半 token”（ECCV 2024） 13
4.4 EAGLE 三部曲：投机解码的自我进化（ICML 2024 / NeurIPS 2025） 14
4.5 MiniCPM-V 与 TinyLLaVA Factory：端侧 MLLM 的配方之争（2024） 14

5 研究前沿（2025–2026） 15

5.1 KV cache 走向极致 15
5.2 视觉 token 压缩：从” 删除” 到” 可恢复” 16
5.3 端侧 MLLM 白热化 16
5.4 多模态 serving 走向成熟 16
5.5 多模态投机解码的再思考 17
5.6 Inference-time scaling 的效率代价 17

6 技术路线对比与权衡 17

7 开放问题与未来方向 19

8 研究机会点评（给入门研究者的建议） 19

9 术语表 20

10 参考文献 21

摘要 多模态大模型（MLLM）的能力边界在过去三年被反复推高，但” 能用” 与” 用得起” 之间始终横亘着一道工程鸿沟：一个 8B 参数的视觉语言模型，仅权重就占用 16 GB 显存，一张高分辨率图像又可展开为上千个视觉 token，使自回归推理的显存与延迟成本进一步数倍于纯文本场景。本报告系统梳理 MLLM 高效化与端侧部署的技术版图。我们从推理过程的两阶段结构（prefill/decode）出发，推导 KV cache 显存公式并指出 decode 阶段的访存瓶颈本质，由此统一理解后续所有技术的动机。随后按四条主线展开：其一，模型压缩——以 GPTQ、AWQ、SmoothQuant 为代表的权重量化，以 SparseGPT、Wanda、LLM-Pruner 为代表的剪枝，以及以 TinyLLaVA Factory 为代表的小模型训练配方；其二，解码加速——从 speculative sampling 到 EAGLE 系列的无损加速范式；其三，多模态特有的视觉 token 压缩——ToMe、FastV、LLaVA-PruMerge 及 2025 年后兴起的” 可恢复压缩”；其四，系统层——vLLM 的 PagedAttention 与 SGLang 的 RadixAttention 所奠定的现代 serving 栈。报告深度剖析 vLLM、GPTQ/AWQ、FastV、EAGLE 三部曲与 MiniCPM-V 五组代表作，并讨论端侧 MLLM（MiniCPM-V、Gemma 3n、SmolVLM、moondream）与 inference-time scaling 带来的效率代价。报告最后给出五条技术路线的对比权衡表、八个开放问题，以及面向入门研究者的算力可行性评估。所有关键数字均来自已核验的论文摘要或原文，文献 ID 经 arXiv 逐一验证。

1 引言：效率在多模态版图中的位置

在本系列报告的多数专题里，我们关心的是” 模型能做什么”：视觉-语言对齐（专题 01）、视频理解（专题 03）、具身决策（专题 06）。本专题关心一个同样本质的问题——**这些能力以什么代价交付**。一个残酷的事实是：2024–2025 年开源旗舰 MLLM 的能力提升，相当程度上是靠” 堆资源” 换来的——更大的 ViT、更高的输入分辨率、更多的训练数据、更长的推理链。而产业落地的真实约束恰恰相反：手机只有 6–12 GB 可用内存，边缘 NPU 只支持 int8/int4 算子，云端单卡的每秒请求数直接决定服务成本。效率方向要解决的，就是在**能力不退化**的前提下，把模型塞进这些约束里。

学习地图（先修概念 → 核心问题 → 关联方向）：

- **先修概念**：Transformer 自注意力及其 $O(n^2)$ 复杂度；自回归解码；浮点数表示（FP16/BF16/INT8）；GPU 的显存层级（HBM 与 SRAM）；LoRA 微调。本报告会按需回顾，不假设读者有系统方向背景。
- **本方向的核心问题**：三个层次——（1）*模型层*：权重与激活能否用更少比特表示？参数能否成块删除而不伤能力？（2）*算法层*：解码能否从逐 token 串行变为并行起草-验证？视觉 token 能否在进入 LLM 之前或之中被压缩？（3）*系统层*：KV cache 如何管理才能让单卡服务更多并发？多模态请求的异构负载（视觉编码 + 文本解码）如何调度？
- **与其他方向的关系**：效率不是孤立专题。视觉 token 压缩直接源于专题 01 的高分辨率策略之争；投机解码的收益高度依赖模型架构（专题 12 的扩散语言模型与之构成替代路线）；agent（专题 07）的长轨迹截图序列让 KV cache 问题雪上加霜；而 inference-time scaling（专题 12 的推理扩展）则站在效率的对立面——用更多计算换智能。理解效率，才能理解这些方向的真实成本结构。

一条值得记住的主线：**这个方向的每一项技术，都可以追溯到一个具体的硬件瓶颈**。量化对抗的是显存容量与显存带宽；投机解码对抗的是 decode 阶段的低算术强度；PagedAttention 对抗的是显存碎片化；视觉 token 压缩对抗的是 $O(n^2)$ 注意力与超长 prefill。抓住” 瓶颈—对

策”的对应关系，比记住每个方法的细节更重要。

还需预先声明本报告的边界：我们讨论**推理效率与部署**，不涉及训练效率（那是另一个同样庞大的话题，本系列专题 11 会覆盖）；我们聚焦于已被广泛验证的主流技术，对二值化、线性注意力等尚未在 MLLM 上大规模验证的路线只作定性提及；报告中的加速比数字均摘自原论文的特定设置（特定模型、硬件、batch），横向比较时请务必注意前提，这也是 §6 对比表只给量级不给排名的原因。

2 基础知识

2.1 推理的两个阶段：prefill 与 decode

自回归 LLM 的一次推理被清晰地分为两段。**Prefill（预填充）**：把整段输入 prompt（对 MLLM 而言是视觉 token + 文本 token）一次性并行送入模型，计算出每个位置的 K、V 并写入缓存，产出第一个输出 token。这一阶段是**计算受限（compute-bound）**的：序列并行度高，矩阵乘足够大，GPU 算力被充分利用。**Decode（解码）**：之后每生成一个 token，都要把整个模型的全部权重从显存读一遍，却只完成一个 token 的前向计算——算术强度（每字节访存对应的浮点运算数）极低，是**访存受限（memory-bound）**的。

对 MLLM 还要加上第三个环节：**视觉编码**。图像先过 ViT 产生视觉 token（compute-bound，但只发生一次），再经连接器投影进 LLM 的词嵌入空间。由此 MLLM 的端到端延迟是三段之和：$T = T_{\text{ViT}} + T_{\text{prefill}} + T_{\text{decode}}$。不同任务的瓶颈完全不同：给一张图配一句短 caption，ViT 与 prefill 占大头；让模型就一张图写千字分析，decode 占大头。这解释了为什么 §5.5 的多模态投机解码研究要先做延迟分解——离开负载画像谈加速是空谈。

这个二分解释了为什么”加速推理”有两个截然不同的战场：prefill 要快，靠的是算子效率（FlashAttention）与减少输入长度（视觉 token 压缩）；decode 要快，靠的是减少访存量（量化、小模型）或让每次权重读取产出多个 token（投机解码）。

2.2 KV cache 显存公式：一行推导读懂所有 KV 优化

Decode 阶段为了避免重复计算历史 token 的注意力，会缓存每一层的 key 与 value——这就是 KV cache。设模型有 L 层，每层 H_{kv} 个 KV 头（GQA 架构下可远小于注意力头数），每头维度 d_h，元素按 b 字节存储（FP16 为 2），则**每个 token 的 KV cache 占用**为：

$$M_{\text{KV/token}} = 2 \times L \times H_{kv} \times d_h \times b$$

以 LLaMA-2-7B 为例（$L = 32$，$H_{kv} = 32$，$d_h = 128$，FP16）：$M = 2 \times 32 \times 32 \times 128 \times 2 = 524288$ 字节，即 **0.5 MB/token**。一段 2048 token 的上下文就占 1 GB；batch 为 8、上下文 4096 时，仅 KV cache 就需要 16 GB——已经超过权重本身（7B 参数 FP16 约 14 GB）。对 MLLM 问题更尖锐：一张高分辨率图像经动态切分可产生数千个视觉 token，视频理解动辄上万 token，KV cache 随图像数量与分辨率**线性膨胀**。

一个容易忽略的细节：公式中的 H_{kv} 是 KV 头数而非注意力头数。多查询注意力（MQA）与分组查询注意力（GQA）让多个 query 头共享同一组 KV，H_{kv} 可从 32 降到 8 甚至 1——KV cache 同比缩小。这就是为什么 2024 年后的新架构几乎清一色采用 GQA：**它是在模型出**

生证明上就写好的 KV 压缩。同理，Gemma 3 的 5:1 局部/全局注意力交错，本质是让五分之四的层只缓存滑动窗口内的 KV，把公式里的" 序列长度" 换成了" 窗口长度"。

这个公式同时指明了四条优化路线，请读者对照后文记忆：（1）减小 b——KV cache 量化（KVQuant）；（2）减小有效 token 数——KV 驱逐与压缩（H2O、StreamingLLM、SnapKV）、视觉 token 剪枝（FastV）；（3）减小 H_{kv}——架构层的 GQA/MQA 与 Gemma 3 的局部注意力；（4）改善显存利用率本身——PagedAttention 的块化管理。一切 KV cache 论文都在这四个变量的乘积上做文章。

2.3 算术强度与 roofline：为什么" 小批量 decode" 天然慢

Roofline 模型告诉我们：一段计算的吞吐上界是 $\min(P, \beta \cdot I)$，其中 P 是峰值算力，β 是显存带宽，I 是算术强度（FLOPs/byte）。Decode 阶段每步要读取全部权重 W 字节，却只做 $O(\text{batch})$ 次向量-矩阵乘，算术强度约等于 batch size——远小于把 GPU 喂饱所需的几百。这就是为什么：

- **量化能线性提速 decode**：权重从 FP16 压到 INT4，每步读取的字节数降为 1/4，访存受限场景下延迟近似同比下降（GPTQ 实测 A100 上 3.25×、A6000 上 4.5× 端到端加速，与理论一致）。
- **投机解码是" 免费" 的并行**：验证 k 个 draft token 与生成 1 个 token 的访存量几乎相同（都只需读一遍权重），却可能一次接受多个 token——本质是把闲置的算力换成吞吐。
- **增大 batch 是最朴素的加速**：直到 KV cache 把显存吃光为止——这又绕回了 §2.2。

2.4 量化基础：从均匀量化到" 难度转移"

量化的目标是把高精度张量 x 映射到低比特整数。最基本的**对称均匀量化**为：

$$\hat{x} = s \cdot \text{clamp}\Big(\text{round}\Big(\frac{x}{s}\Big),\ -2^{b-1},\ 2^{b-1}-1\Big), \qquad s = \frac{\max|x|}{2^{b-1}}$$

误差来自 round 操作，理论信噪比随比特数指数恶化。LLM 量化的真正困难不在权重——**激活中存在少量幅值极大的 outlier 通道**（LLM.int8() 的实证发现），它们把缩放因子 s 撑大，使其余 99% 的数值分辨率被浪费。后文三条经典路线正是对这一困难的不同回答：LLM.int8() 把 outlier 通道拆出来用 FP16 单独算；SmoothQuant 通过数学等价变换把激活的 outlier" 难度" 转移到权重上；AWQ 则反其道而行——只用激活分布**找出**那 1% 的 salient 权重通道并予以保护。同一个痛点，三种哲学。

两个工程细节值得记住。其一，**分组量化（group-wise quantization）**：不是整个张量共享一个缩放因子，而是每 64 或 128 个权重一组各用各的 s——用少量额外存储换回精度的大幅提升，GPTQ/AWQ 的 4-bit 实践全部依赖于此。其二，**PTQ 与 QAT 的分工**：训练后量化（PTQ）只需几百条校准样本、小时级成本，是部署的默认选项；量化感知训练（QAT）把量化噪声纳入训练循环，精度更高但成本相当于一次微调，只在新模型发布前的" 官方量化" 中划算。本报告讨论的方法全部属于 PTQ 阵营。

2.5 投机解码的数学保证：为什么”起草-验证”是无损的

speculative sampling 的精妙之处值得单独一节。设目标模型分布为 p，draft 模型分布为 q。草稿阶段用 q 自回归采出 k 个 token，然后目标模型**一次前向**并行算出这 k 个位置各自的 $p(x \mid \text{prefix})$——注意自回归的依赖性在这里不构成障碍，因为验证时整段前缀已知，可以像 prefill 一样并行。接着从左到右逐个裁决：以概率 $\min(1, p(x)/q(x))$ 接受 draft token x；一旦拒绝，就从修正分布 $\text{norm}(\max(0, p-q))$ 重采一个 token 并终止本轮。可以证明，这样得到的输出序列与直接从 p 采样**同分布**——加速而不改变模型的任何行为，这就是”无损”的精确含义。

这个框架同时给出了收益公式：每轮的期望接受 token 数取决于 p 与 q 的接近程度，而每轮成本约等于”目标模型一次前向 + draft 模型 k 次小前向”。由于 decode 访存受限，目标模型验证 k 个 token 与生成 1 个 token 的延迟几乎相同——**闲置的算力被换成了吞吐**。由此立刻读出两个设计准则：draft 模型要足够小（起草便宜）又要足够像目标模型（接受率高），这对矛盾的张力催生了 EAGLE 系列的全部创新（§4.4）；而在大 batch 场景算力不再闲置，投机解码的收益自然衰减——EAGLE-3 论文在 SGLang 中 batch 64 仍有 1.38× 吞吐提升，这个数字的分量要放在这个背景下理解。

2.6 算子层的地基：FlashAttention 与”显存读写比 FLOPs 更贵”

在讨论一切压缩技术之前，必须先纠正一个从教科书继承来的错觉：注意力的瓶颈不是 $O(n^2)$ 的浮点运算量，而是 $O(n^2)$ 的**显存读写量**。标准实现把 $QK^\top$ 与 softmax 结果两个 $n \times n$ 矩阵完整写入 HBM 再读回，而 HBM 带宽比 SRAM 慢一个数量级以上。FlashAttention（2022）的解法是借鉴分块矩阵计算：把 Q、K、V 切成能塞进 SRAM 的 tile，在块内完成注意力并用**在线 softmax**（维护运行最大值与归一化因子）逐步修正结果，全程不物化 $n \times n$ 矩阵——数学上是**精确**注意力，不是近似。收益是注意力的 HBM 访问量从二次降为线性，训练与推理全面提速，且序列越长收益越大。FlashAttention-2（2023）进一步削减非 matmul 运算占比、改进 GPU warp 间的工作划分，再获约 2 倍提速。FlashAttention 对效率方向的意义是范式性的：**它证明了”理解硬件层级”本身可以比任何算法近似带来更大收益**，这一哲学后来被 PagedAttention、KV cache 量化反复继承。今天没有任何现代训练或 serving 栈不使用它——它是本报告所有系统数字的隐形前提。

2.7 剪枝与蒸馏：两种”变小”的哲学

压缩模型有两大范式。**剪枝（pruning）**假设大模型存在冗余参数，删之即可：非结构化剪枝把个别权重置零（如 SparseGPT、Wanda），压缩率高但得到的稀疏矩阵在通用 GPU 上没有实测加速，收益主要在存储；结构化剪枝整块删除通道、注意力头甚至层（如 LLM-Pruner、SliceGPT），压缩率较低但每一刀都直接转化为更小的矩阵乘，加速真实。剪枝方法论的核心问题是**重要性度量**：凭什么判断某参数可有可无？幅值、梯度、二阶信息（Hessian）、激活统计各有拥趸，§4 与 §6 会展开。**蒸馏（distillation）**则是另一条路：不修改大模型，而是让小模型（学生）模仿大模型（教师）的输出分布或中间特征。对 MLLM 而言，蒸馏常以更朴素的形式出现——用大模型生成高质量指令数据直接训练小模型，TinyLLaVA Factory 与 MiniCPM-V 的配方（§4.5）本质上都是”数据蒸馏”的胜利。两条哲学的分野在于：剪枝相信”大象可以瘦身”，蒸馏相信”小马可以学大象的舞步”。实践中二者并不互斥：LLM-Pruner 剪后用 LoRA 恢复，QLoRA 在

4-bit 量化权重上微调（4-bit NormalFloat 量化 + 双重量化 + 分页优化器使单张 48GB 卡可微调 65B 模型），都是混合策略的代表。

2.8 多模态的特殊性：视觉 token 的数量级

纯文本场景里，一段对话的 token 数以百到千计；而在 MLLM 中，**一张图像就是一篇长文**。以 patch 大小 14 的 ViT 为例：448×448 的图像产生 $(448/14)^2 = 1024$ 个视觉 token；LLaVA-1.5 使用 336 分辨率得到 576 token；一旦引入动态高分辨率切分（AnyRes、Qwen2-VL 的 naive dynamic resolution），单图 token 数轻松破两千，视频输入再乘以帧数。这些 token 与文本平等地参与 LLM 每一层的注意力，使 prefill 计算量与 KV cache 同时爆炸。更微妙的是，**视觉 token 存在文本所没有的空间冗余**——相邻 patch 常常描绘同一片天空、同一块墙面。这个观察支撑起整个”视觉 token 压缩”子领域（§3、§4.3、§5.2），也是多模态效率研究区别于纯文本效率研究的标志性问题。

2.9 端侧硬件约束：为什么是 int4 而不是别的

端侧部署的所有决策都被三组数字支配：**内存容量**（手机给单个 App 的可用内存通常只有 4–8 GB，一个 FP16 的 8B 模型根本放不进去，int4 后约 4.5 GB 才勉强可行）；**内存带宽**（旗舰手机 LPDDR 约 60–100 GB/s，是数据中心 GPU 的 1/20 到 1/40，decode 阶段每秒能读几遍权重直接决定 token 生成速度的上限——粗算，4.5 GB 的 int4 模型在 80 GB/s 带宽下理论上限约 17 token/s）；**算子支持**（移动 NPU 的 int8/int4 矩阵乘成熟，但 FP16 算力与算子覆盖率参差，混合精度常常退化回 CPU）。这三组数字解释了端侧技术栈的收敛方向：权重 int4 是容量与带宽的双重必然；W4A16 权重反量化方案（而非 W8A8）流行是因为激活 FP16 对精度更稳、且 decode 的瓶颈在权重读取不在计算；而 Gemma 3n 的 Per-Layer Embedding（把每层嵌入表卸载到更快的存储、按需加载）则是对”嵌入表在 2B 小模型里占比过大”这一端特有问题（内存占比）的直接回应。

3 经典方法谱系

效率方向的技术积累可以按”问题被发现与解决”的顺序串成五条时间线。读这些时间线时请注意一个节奏：**几乎每隔一年，瓶颈就下移一层**——2022 年瓶颈在算子（FlashAttention），2023 年在显存管理（vLLM）与权重比特（GPTQ/AWQ），2024 年在序列长度（视觉 token 压缩、KV 驱逐），2025 年在解码范式（EAGLE-3）与端侧架构（Gemma 3n）。瓶颈不会消失，只会转移。

算子与系统层（2022–）。FlashAttention（2022）用 tiling 与在线 softmax 把注意力的 HBM 读写从二次降为线性，成为一切现代训练与推理栈的算子地基；FlashAttention-2（2023）通过减少非 matmul FLOPs 与改进 warp 分工再提速约 2 倍。2023 年是 serving 系统的分水岭：vLLM 的 PagedAttention 借鉴操作系统虚拟内存思想管理 KV cache，SGLang 的 RadixAttention 用基数树自动复用共享前缀，二者共同定义了”LLM serving”这个学科。

量化与剪枝（2022–2024）。LLM.int8()（2022）首次指出激活 outlier 问题；同年 GPTQ 把二阶信息引入一次性权重量化，SmoothQuant 完成 W8A8 的难度转移。2023 年 AWQ 提出激活感知的 salient 通道保护，SpQR 用稀疏 outlier + 低比特主体逼近无损压缩，QLoRA 把

4-bit 引入微调侧。剪枝线几乎同步：SparseGPT 一次性稀疏回归、Wanda 的” 幅值 × 激活” 极简基线、LLM-Pruner 的结构化剪枝、SliceGPT 基于 RMSNorm 计算不变性的整行整列删除，覆盖了从非结构化到结构化的完整频谱。

解码加速（2023–2025）。speculative sampling（2023，DeepMind/Google 各自独立提出）确立” 小模型起草、大模型并行验证、输出分布不变” 的范式；Medusa（2024）用多个解码头实现自起草；EAGLE（2024）把起草提升到 feature 层并引入树形验证，EAGLE-2（2024）发现接受率的上下文依赖并做动态草稿树，EAGLE-3（2025）放弃特征约束、改用多层特征融合与 training-time test，把加速比推至 6.5×。

蒸馏与小型化（2023–）。与压缩并行的一条线是” 重新训练小模型”：MobileVLM（2023）为移动端设计轻量 projector（LDP），在骁龙 888 上实现实时推理；MoE-LLaVA（2024）把混合专家引入小 MLLM，用稀疏激活换取容量；TinyLLaVA Factory（2024）系统化小模型训练配方；Align-KD（2024）专攻跨模态对齐知识的蒸馏；MiniCPM-V（2024）则把端侧能力推至 GPT-4V 级。这条线的共识逐渐清晰：**小模型的差距主要不在容量而在数据与训练策略**。

视觉 token 压缩（2022–）。ToMe（2022）最早在 ViT 内部做 token 合并——它发现 Transformer 的 token 表示随层数加深逐渐冗余，提出用二分软匹配（bipartite soft matching）逐层合并相似 token，不改权重即可为 ViT 提速；FastV 与 LLaVA-PruMerge（2024）把战场移到 MLLM 内部，分别发现” 第二层后视觉 token 注意力骤降” 与”[CLS] 注意力天然稀疏” 两个可操作的冗余信号；LLaMA-VID 把视频帧压到 2 token/帧；工程侧的 pixel shuffle（InternVL 2.5）与 2×2 平均池化（Qwen2-VL）则成为开源旗舰的标配。

3.1 关键论文一览表

论文	年份	核心思想	架构/系统要点	备注
FlashAttention (2205.14135)	2022	IO 感知的精确注意力	tiling + 在线 softmax	算子层地基
LLM.int8() (2208.07339)	2022	outlier 通道混合精度分解	向量级量化 + FP16 outlier	首次指出激活 outlier
GPTQ (2210.17323)	2022	二阶信息逐层权重量化	OBQ 改进 + 分组量化	社区事实标准
SmoothQuant (2211.10438)	2022	激活难度数学转移到权重	per-channel 等价缩放	W8A8 无损
Speculative Sampling (2302.01318)	2023	小模型起草、并行验证	拒绝采样保分布	无损加速范式开启
SparseGPT (2301.00774)	2023	一次性稀疏回归剪枝	逐层 Hessian 近似	50–60% 稀疏免重训
AWQ (2306.00978)	2023	激活感知保护 1% salient 通道	等价缩放替代混合精度	首个覆盖多模态 LM

论文	年份	核心思想	架构/系统要点	备注
vLLM / PagedAttention (2309.06180)	2023	OS 分页管理 KV cache	block 表 + copy-on-write	serving 事实标准
StreamingLLM (2309.17453)	2023	attention sink + 滑窗	保留首 token	无限长流式输入
SGLang / RadixAttention (2312.07104)	2023	前缀 KV 自动复用	基数树缓存 + 结构化 DSL	agent 工作流加速
Wanda (2306.11695)	2023	幅值 × 输入范数剪枝	无需任何重建	极简强基线
LLM-Pruner (2305.11627)	2023	梯度重要性结构组剪枝	剪后 LoRA 快速恢复	结构化代表作
EAGLE (2401.15077)	2024	feature 层自起草	时移 token 消不确定性 + 树验证	被 vLLM/SGLang 集成
Medusa (2401.10774)	2024	多解码头并行预测	冻结主干	易部署
KVQuant (2401.18079)	2024	KV cache <4bit	per-channel key / per-token value	千万级上下文
SliceGPT (2401.15024)	2024	RMSNorm 不变性旋转后删行列	结构化 dense 压缩	25% 参数可删
FastV (2403.06764)	2024	第二层后视觉 token 冗余	即插即用剪视觉 token	FLOPs -45%
LLaVA-PruMerge (2403.15388)	2024	[CLS] 注意力自适应剪 + 合并	关键相似度聚类补信息	平均 14× 压缩
SnapKV (2404.14469)	2024	观测窗口注意力筛 KV	prompt 末端窗口投票	长上下文即插即用
TinyLLaVA Factory (2405.11788)	2024	小 MLLM 训练配方	模块化工厂框架	数据配比 > 尺寸
MiniCPM-V (2408.01800)	2024	端侧 GPT-4V 级 MLLM	高分辨率切分 + 数据/对齐工程	8B 上手机
Qwen2-VL (2409.12191)	2024	任意分辨率 + token 池化	naive dynamic resolution + M-RoPE	2×2 压缩标配化
InternVL 2.5 (2412.05271)	2024	ViT 渐进放大 + pixel shuffle	视觉 token 减半	开源性能标杆

论文	年份	核心思想	架构/系统要点	备注
DeepSeek-VL2 (2412.10302)	2024	MoE 视觉语言模型	激活 4.5B + 动态切图	低成本高能力
EAGLE-3 (2503.01840)	2025	放弃特征约束 + training-time test	多层特征融合起草	加速至 6.5×
Gemma 3 / 3n (2503.19786)	2025	局部/全局注意力交错省 KV	MatFormer + PLE（3n）	手机端 2B/4B 有效参数

4 代表作深度剖析

4.1 vLLM 与 PagedAttention：把操作系统请进 GPU（SOSP 2023）

研究问题。高吞吐 serving 的前提是尽可能大的 batch，但 KV cache 显存成为瓶颈。2023 年之前的系统（FasterTransformer、Orca）为每个请求分配一段**连续**的 KV 显存，并按最大可能长度预留——这带来两类浪费：内部碎片（实际生成短于预留）与外部碎片（不同长度请求之间无法拼合），论文实测显存浪费可达 60–80%。问题不是 KV 太大，而是**管理太笨**。

关键洞察。KV cache 的显存管理问题与操作系统管理进程内存的问题同构：变长、动态增长、需要共享（parallel sampling、beam search、共享 system prompt）。那么答案也应当同构——**分页（paging）**。

技术方法。PagedAttention 把每个序列的 KV cache 切成固定大小的 block（每块装若干 token 的 KV），通过一张 block table 做逻辑块到物理块的间接映射，物理块按需分配、无需连续。注意力 kernel 被改造为按块取数。由此得到两个杀手级特性：（1）**近零浪费**：碎片只剩每个序列最后一个未填满的块；（2）**近乎免费的共享**：beam search 或 parallel sampling 的公共前缀只需存一份物理块，block table 引用计数 + copy-on-write 处理分叉。在此之上，vLLM 实现了 continuous batching——不等整个 batch 完成，每生成一步就把已结束的请求换出、新请求换入，把 GPU 占用率拉满。

实验结果。论文报告在相同延迟水平下，vLLM 相对 FasterTransformer 与 Orca 的吞吐提升 **2–4 倍**，且序列越长、模型越大、解码算法越复杂，优势越明显（这正是碎片浪费最严重的场景）。两个评测细节值得品味：其一，比较基线 Orca 已是当时最强的 serving 系统（首创 continuous batching），说明收益来自显存管理而非调度；其二，优势随并行采样数扩大——beam search 与 parallel sampling 下共享前缀的 copy-on-write 机制开始发力，这预示了后来 agent 时代” 一请求多分支” 负载的形态。

批判性分析。优点：PagedAttention 不损失任何精度、不改变任何模型行为，是纯系统收益，且思想一经提出便成为行业标准——2025 年的 vLLM V1 已原生支持 VLM（视觉编码器分离调度、多模态 prefix cache）。局限：分页只解决” 放得巧”，不解决” 放不下”——KV 总量仍随序列线性增长，需要与量化、驱逐技术叠加；此外 block 大小是碎片与 kernel 效率的权衡参数，对极端长度分布仍需调优。可复现性极佳：代码开源，社区生态庞大。

延伸阅读：SGLang 与 RadixAttention。PagedAttention 解决单个请求内部的显存浪费，

但真实负载中大量请求**共享前缀**——同一个 system prompt、同一套 few-shot 示例、agent 循环里反复出现的历史。SGLang（NeurIPS 2024）把 KV cache 组织成一棵基数树（radix tree），新请求到来时自动匹配树中最长已缓存前缀，命中部分零计算；配合面向结构化生成的前端 DSL 与压缩有限状态机（加速 JSON 等受限解码），在 agent 控制、多轮对话、RAG 流水线等复杂工作负载上取得最高 6.4× 的吞吐提升。RadixAttention 与 PagedAttention 是互补的：前者管"请求之间"的复用，后者管"请求之内"的布局。2025 年两大系统殊途同归地原生支持 VLM——视觉编码器被拆成独立调度单元，图像的 KV 同样进入 prefix cache，"同一张图问十个问题"只需编码一次。

一句话评价：近五年对 LLM 落地影响最大的单篇系统论文之一，评级 ★★★★★。

4.2 GPTQ 与 AWQ：量化双雄的两种世界观（ICLR 2023 / MLSys 2024）

研究问题。训练后量化（PTQ）在 CV 模型上早已成熟，但搬到百亿参数的生成式 Transformer 上集体失效：4-bit 下困惑度飙升。要压缩到单卡可装（175B 模型从 ~350 GB 压到 ~90 GB），误差控制必须精确到"每个权重对最终输出的贡献"这一粒度。

GPTQ 的路线：相信二阶信息。GPTQ 继承 OBQ（Optimal Brain Quantization）的思想：逐层求解，每量化一个权重，就用该层 Hessian 的逆（由少量校准数据的二阶统计近似）调整其余未量化权重以补偿误差，使层输出的平方误差最小化。其工程贡献同样关键：按固定顺序逐列量化 + Cholesky 技巧摊销 Hessian 求逆 + lazy batch 更新，把复杂度降到 175B 模型约 4 个 GPU 小时即可完成。结果：3/4-bit 下相对 FP16 基线的精度损失可忽略，175B 模型首次能在单张 GPU 上做生成式推理，端到端加速 3.25×（A100）/ 4.5×（A6000）。

AWQ 的路线：相信激活分布。AWQ 的出发点是一个更朴素的观察：不是所有权重生而平等——仅保护 1% 的 salient 权重就能大幅降低量化误差；而判断哪些权重 salient，应看激活的幅值分布而非权重自身（因为大激活通道上的权重误差会被放大）。但把 1% 通道存成 FP16 会造成混合精度的硬件低效，AWQ 的巧思是用等价变换替代混合精度：把 salient 通道的权重按系数 $s > 1$ 放大、对应激活除以 s（数学上逐元素等价），使这些通道的相对量化误差缩小约 s 倍。缩放系数由离线激活统计决定，不依赖反向传播与重建，因此不会过拟合校准集——这使 AWQ 成为首个在指令微调模型与多模态模型上都表现优秀的量化方法。配套的 TinyChat 推理框架在桌面与移动 GPU 上相对 HuggingFace FP16 实现提速超 3×，并把 70B 模型搬上了移动 GPU。

第三条路线补记：SmoothQuant 与 W8A8。GPTQ/AWQ 都是 weight-only 方案，省带宽不省算力。要吃到 GPU/NPU 的 int8 矩阵乘算力，必须同时量化激活（W8A8），而激活 outlier 正是拦路虎。SmoothQuant（ICML 2023）的观察极为对称：激活的 outlier 集中在少数固定通道，而权重的分布平坦——既然如此，用一个 per-channel 的等价变换 $Y = (X\mathrm{diag}(s)^{-1})(\mathrm{diag}(s)W)$ 把激活按通道缩小、权重对应放大，量化难度就从激活侧转移到了权重侧。单个平滑系数 $\mathrm{diag}(s)$ 还可以被吸收进前一层算子，运行时零开销。这一"难度转移"思想与 AWQ 的"salient 通道缩放"在数学上是同一枚硬币的两面：AWQ 放大权重保护少数通道，SmoothQuant 缩小激活压平 outlier。

批判性分析。GPTQ 胜在有严格的误差界与极强的 2–3 bit 极限表现，但逐层重建过程较重、对校准数据分布更敏感；AWQ 胜在轻量、泛化好、天然适配多模态与端侧（这解释了它在 llama.cpp/TensorRT-LLM 生态中的流行）。二者的共同局限：都是 weight-only（W4A16），decode 阶段仍需把权重反量化为 FP16 计算，算力本身并未节省——要吃到 int 算子的红利需 W8A8

（SmoothQuant）或更激进的方案，而那又受限于激活 outlier。可复现性：两者均开源，社区复现无数。

一句话评价：一个是" 用数学把误差推平"，一个是" 用先验把误差绕开"，合读即得 PTQ 方法论全景，各 ★★★★★。

4.3 FastV：发现" 图像在第二层之后只值一半 token"（ECCV 2024）

研究问题。视觉 token 压缩的前作（ToMe 等）大多在视觉编码器内部做文章，而 MLLM 的真实算力大头在 LLM 主干：576 个视觉 token 参与 LLaVA-1.5-13B 的每一层注意力。问题是：LLM 真的在每一层都同等需要这些视觉 token 吗?

关键洞察。FastV 先做诊断后开药方：可视化 LLaVA-1.5、QwenVL-Chat、Video-LLaVA 各层的注意力分布后发现一个极强的规律——浅层（约第 2 层之前）视觉 token 被充分关注，深层之后注意力急剧塌缩到少数系统/指令 token 上。LLM 在浅层已完成" 看图"，深层主要在" 读题"。视觉 token 的深层注意力计算是极端低效的。

技术方法。FastV 是即插即用的两参数方法：选一个剪枝层 K（如第 2 层），在该层用注意力分数给视觉 token 排序，保留 top-R%（如 50%），其余直接丢弃，后续层照常前向。不训练、不改权重、不换架构。

实验结果。在 LLaVA-1.5-13B 上以 50% 保留率剪掉一半视觉 token，FLOPs 降低 45%，而在一系列图像与视频理解基准上精度几乎不变；论文进一步展示其帕累托效率——把 13B 模型的计算预算压到低于 7B 模型，性能仍优于后者。FastV 还可与 KV cache 策略（如 SnapKV）叠加。

对照阅读：LLaVA-PruMerge 的" 剪 + 补"。FastV 删掉的信息彻底消失，PruMerge（2024）则多问了一步：被剪的 token 能不能" 托孤"? 它利用视觉编码器中 [CLS] token 对各 patch 注意力分布天然稀疏的性质，自适应地选出重要 token；随后对被剪掉的 token 按 key 相似度聚类，把每簇的信息合并进与之最相似的保留 token 中——删的是位置，留的是信息。在 LLaVA-1.5 上平均压缩视觉 token 14 倍仍保持可比的 VQA 与推理性能。从 FastV 到 PruMerge 再到 § 5.2 的" 可恢复压缩"，能清晰看到这个子领域的问题意识从" 删多少" 进化到" 删掉的去哪了"。

批判性分析。优点是优雅到极致的发现型工作：一个可复现的诊断（注意力塌缩）直接导出一个零成本部署技巧，且揭示的" 浅层看、深层读" 相变对理解 MLLM 内部机制有独立价值。局限：统一在第 K 层一刀切是静态策略，对 OCR、文档、细粒度定位等" 深层仍需看图" 的任务，保留率必须调高，论文给出的帕累托曲线实际上承认了这一点；注意力分数作为重要性指标也被后续工作证明并非最优（与 2025 年" 校准" 研究的发现一致）。与 § 2.5 的 token 数量级对照，FastV 的 45% FLOPs 削减在 LLaVA-1.5（576 视觉 token）上成立；对 AnyRes 类高分辨率输入（2000+ token），由于注意力成本随视觉 token 数二次增长，同样的剪枝率带来的绝对收益更大——但风险同样放大，因为高分辨率输入恰恰多用于细粒度任务。可复现性：代码开源，实现只需几十行。

一句话评价：用一条注意力曲线开创了一个子领域，多模态效率必读，★★★★★。

4.4 EAGLE 三部曲：投机解码的自我进化（ICML 2024 / NeurIPS 2025）

研究问题。speculative sampling（2023）证明了无损加速的可能：小 draft 模型自回归起草 k 个 token，大模型一次前向并行验证，用拒绝采样保证输出分布与目标模型完全一致。但它留下一个工程痛点：为每个目标模型训练并部署一个独立 draft 模型，成本与维护负担都不小。能否让目标模型" 自己起草自己"?

EAGLE：feature 层起草。EAGLE 的两个洞察层层递进：（1）在目标模型倒数第二层特征（second-to-top-layer feature）空间做自回归，比在 token 空间容易得多——特征是连续、信息丰富的；（2）但特征级自回归存在内在不确定性：同一个特征序列可对应不同采样结果。解法是向 draft 模型额外输入时移一步的 token 序列，把采样结果的不确定性显式喂回去。draft 模型只有约一层 Transformer 的体量，代价极小；配合树形注意力（tree attention）一次验证多条候选路径。在 LLaMA2-Chat 70B 上取得 2.7–3.5× 延迟加速、吞吐翻倍，且分布无损。

EAGLE-2：上下文感知的动态草稿树。EAGLE-2 发现一个此前被隐含否定的现象：draft token 的接受率不只取决于它在树中的位置，还取决于上下文；而 EAGLE 的 draft 模型恰好是良校准的——其置信度近似接受率。于是用 draft 置信度在线构建动态草稿树，把验证算力花在" 最可能被接受的分支" 上。加速比升至 3.05–4.26×，比 EAGLE-1 再快 20–40%，依旧无损。

EAGLE-3：拥抱数据 scaling。EAGLE-3 反问自己一个尴尬问题：为什么给 draft 模型更多训练数据，EAGLE 的收益增长却很有限？答案是瓶颈在特征预测约束——要求 draft 复现目标模型的顶层特征，等于给它套了一个与大模型能力无关的目标。EAGLE-3 放弃特征预测、回归直接预测 token，并把输入从顶层特征改为多层特征融合；关键机制 training-time test 让 draft 在训练时就模拟多步起草的暴露偏差。结果：加速比至 6.5×（较 EAGLE-2 约 1.4×），且随数据扩展持续获益；在 SGLang 中 batch 64 时吞吐提升 1.38×——注意大 batch 下投机解码收益通常被稀释（算力不再闲置），EAGLE-3 的这个数字含金量很高。

与 Medusa 的对照。同期的 Medusa（2024）走了另一条自起草路线：冻结主干，在原模型上训练多个解码头，每个头负责预测未来第 i 个 token，再用树状注意力验证。Medusa 的优势是部署简单（不需要单独的 draft 网络）、对原模型零侵入；EAGLE 的优势是 draft 自回归（每步起草可以利用上一步的 draft 状态），单步外推能力更强，因此加速比上限更高。2025 年的工业实践基本收敛到 EAGLE 系，但 Medusa 的" 多头并行预测" 思想在单步大跨度场景（如代码补全的模板化续写）仍有一席之地。

批判性分析。三部曲是" 洞察驱动迭代" 的教科书：每一代都精确指认上一代的隐含假设（token 空间起草、静态树、特征约束）并将其证伪。局限：需要针对每个目标模型训练 draft（尽管代价远小于独立 draft 模型）；对高度随机或低置信任务（如采样温度高时）接受率下降；多模态场景下视觉前向占比改变了收益结构（见 § 5.5）。可复现性：开源且已被 vLLM/SGLang 原生集成。

一句话评价：把投机解码从" 学术巧思" 打磨成" 工业标配" 的系列，整体 ★★★★★。

4.5 MiniCPM-V 与 TinyLLaVA Factory：端侧 MLLM 的配方之争（2024）

研究问题。GPT-4V 级能力需要多大模型？2024 年之前的共识是" 至少几十 B"。MiniCPM-V 与 TinyLLaVA Factory 从两个相反的角度挑战这个共识：前者证明 8B 可以上手机并对标 GPT-4V，后者证明 <4B 模型在正确配方下能逼近大模型。

MiniCPM-V：数据与对齐工程的极致。MiniCPM-Llama3-V 2.5（8B）的三个支柱：（1）高分辨率切分——按 LLaVA-UHD 思路把任意长宽比图像切成多张子图编码，支持 180 万像素感知与强 OCR；（2）数据工程——大规模高质量图文对与多语言（30+ 语种）数据配比；（3）对齐工程——RLAIF-V 等降低幻觉的策略。结果：在 OpenCompass 11 基准综合评测上超过 GPT-4V-1106、Gemini Pro 与 Claude 3，且可部署于手机端。论文摘要中那句判断值得引用为本方向的座右铭：达到可用（GPT-4V 级）性能所需的模型尺寸正在快速下降，与端侧算力的增长形成双向奔赴。

TinyLLaVA Factory：把" 配方" 变成第一公民。如果说 MiniCPM-V 给出了一份冠军配方，TinyLLaVA Factory 则系统回答了" 配方里什么最重要"。它以工厂模式把小型 MLLM（<4B）的组件（视觉编码器、连接器、小 LLM、训练策略）全部模块化，并用受控实验比较设计选择。其最重要的实证结论是：数据配比与设计选择对最终性能的影响大于模型尺寸本身——1–3B 的小模型在恰当配方下可逼近 7B 模型。这条结论有两层操作性含义：其一，预训练阶段的视觉-语言对齐数据（caption 类）与指令微调阶段的任务数据之间存在非平凡的最优配比，不是越多越好；其二，小 LLM 骨干的选择（Qwen、Phi、StableLM 等）对上限的影响超过视觉塔与连接器的组合差异。对想复现的读者，这套框架的全部组件可自由替换，代码库即实验报告。

批判性分析。MiniCPM-V 的意义在于把端侧 MLLM 从" 能跑" 推进到" 好用"，但 8B 上手机仍需配合 int4 量化（见 §4.2），其"GPT-4V 级" 结论限定于特定基准组合，不应外推为全面对等；TinyLLaVA Factory 的价值在方法论——它告诉你复现与消融该做什么实验，但其结论（数据 > 尺寸）的适用边界在小模型区间，不能推出" 大模型不需要数据工程"。两者代码与权重均开放，是入门者复现端侧 MLLM 的最佳起点。

工程注记。把 MiniCPM-V 真正放进手机还需要过量化这一关：OpenBMB 官方发布过 int4 版本，配合 GGUF/llama.cpp 生态可在消费级设备运行；而高分辨率切分策略带来的视觉 token 膨胀，则依靠切分数量上限与连接器侧的感知器重采样（perceiver resampler）控制预算。这个" 架构给预算、量化给显存、数据给能力" 的三位一体，是端侧 MLLM 的标准作业流程，后来的 Gemma 3n、SmolVLM 莫不如此。

一句话评价：一个给出终点证明，一个给出路线图，端侧方向的双子星，各 ★★★★☆。

5 研究前沿（2025–2026）

2025 年之后，效率方向的重心发生了三个可辨识的迁移：从" 单次推理" 转向" 工作流"（agent 长轨迹、多轮工具调用成为主要负载）；从" 模型" 转向" 系统"（serving 栈的架构创新比单点算法创新带来更多收益）；从" 省算力" 转向" 省能耗"（端侧评价指标从延迟扩展到焦耳/请求）。以下六个子方向按此脉络组织。

5.1 KV cache 走向极致

先回顾一下光谱：KV cache 优化从" 架构层"（GQA/MQA 减少 KV 头，Gemma 3 用 5:1 局部/全局注意力交错让大部分层不缓存全局 KV）到" 系统层"（PagedAttention 的块化管理）再到" 数据层"（量化与驱逐），2025 年的创新集中在最后一层。

KV cache 优化正从" 省着用" 进入" 精打细算" 阶段。KVQuant（ICML 2024）用 per-channel key / per-token value 的非均匀定制策略把 KV 压到 4 bit 以下，使单卡千万 token 上下文成为可能；驱逐策略方面，H2O 的 heavy-hitter 假设、StreamingLLM 的 attention sink、SnapKV 的观测

窗口投票正在被一个统一的注意力统计理论收编（2607.24667 试图统一 sink 理论与驱逐策略）。更新的工作把粒度继续推向极限：TriAxialKV（2605.17170）面向 agentic 长上下文做超低比特 KV 量化，2604.21335 则探索亚 token 级的路由式压缩。与此同时，系统社区开始讨论 KV 的"成本归属"——当 agent 工作流共享大量前缀，KV 不再免费，控制面需要把缓存成本显式计费（2609.23130、2609.24991）。

5.2 视觉 token 压缩：从"删除"到"可恢复"

2024 年的范式是"找冗余、做减法"；2025 年的范式转向可恢复、可路由的压缩。直觉很直接：FastV 式的一刀切在 OCR、grounding 等任务上翻车，说明被删的 token 并非无用、只是"此时无用"。于是 Reroute, Don't Remove（2606.12412）主张把被剪 token 的信息重路由进保留 token 而非丢弃；E2S-Pruner（2608.23253）等探索任务条件化的可恢复剪枝；2604.12035 则揭示剪枝决策与后续校准的耦合关系。token compression 综述（2507.20198）将方法整理为 merge / prune / resample / spatial pooling 四类。工程侧，pixel shuffle（InternVL 2.5，把空间 2×2 邻域拼入通道维，视觉 token 减半）与 2×2 池化（Qwen2-VL）已被开源旗舰默认采用；视频侧 LLaMA-VID 的 2 token/帧代表了"上下文压缩"的激进极点。"先压缩再进 LLM"在 2025 年成为事实标准，研究热点随之转向"压缩的信息论下界与任务自适应"：给定任务分布，压缩率与性能损失的相变点在哪里？同一个模型能否对 OCR 请求保守、对闲聊请求激进？这些问题把 token 压缩从工程技巧提升为学习问题——压缩器本身可学习、可微、可与下游任务联合优化，是 2026 年值得跟踪的走向。

5.3 端侧 MLLM 白热化

2025 年端侧赛道形成完整梯队：Gemma 3（2503.19786）以 5:1 的局部/全局注意力交错大幅削减 KV cache，覆盖 1B–27B；其移动端变体 Gemma 3n（2025-05，官方模型卡）用 MatFormer 嵌套结构 + Per-Layer Embedding（PLE）把嵌入表从 HBM 挪到片上内存，实现 2B/4B 有效参数在手机运行音频 + 图像理解——"名义参数 > 驻留参数"是端侧架构的新范式。开源社区侧，SmolVLM（Hugging Face，模型卡发布）与 moondream（1.9B，无 arXiv 论文，开源发布）把可用 VQA/caption/检测能力压到 2B 以下；MiniCPM-V/o 系列持续迭代。值得单独一提的是两个"小而美"的极端样本：moondream（1.9B，开源权重，无 arXiv 论文）证明了 caption/VQA/目标检测三合一的能力可以压进 2B 以内，成为边缘开发者的宠儿；SmolVLM（Hugging Face 模型卡发布）则把完整训练配方与数据开源，延续了 TinyLLaVA Factory"配方公开"的传统。这两个项目都没有论文，但它们定义了端侧 MLLM 的事实下限——低于这个尺寸，视觉理解能力开始出现不可接受的退化。部署形态研究也在细化：2609.25415 系统比较了云端、边缘与分割推理（split inference）在无人机场景的能耗-延迟权衡。

5.4 多模态 serving 走向成熟

2025 年 vLLM V1 与 SGLang 均原生支持 VLM：视觉编码器与语言解码被分离调度（编码是 compute-bound 的偶发突发，解码是 memory-bound 的持续负载，资源画像完全不同），多模态 prefix cache 让"同一张图问多个问题"近乎免费。MoE 架构进入 serving 视野——DeepSeek-VL2（激活 4.5B）证明稀疏激活能以远低于 dense 模型的推理成本达到同等理解能力，但 MoE 的专家路由也给负载均衡带来新课题。更深层的趋势是 disaggregated serving：prefill 与 decode 拆到

不同硬件池，各自按需扩缩，多模态工作负载（重 prefill、轻 decode）是这一架构的最大受益者。与之配套的是 chunked prefill——把超长 prefill 切块与 decode 步交错执行，避免一个多图请求阻塞整个 batch 的解码。对读者判断趋势有用的一个信号是：2025 年的 serving 系统发布 notes 里，" 多模态" 已从" 支持列表的一项" 变成" 架构设计的前提"——prefix cache 粒度、调度器目标函数、显存分配器都在按" 请求 = 异构模态片段的序列" 这一抽象重写。

5.5 多模态投机解码的再思考

投机解码能照搬到 MLLM 吗？2608.20743 以" 综述 + 实证诊断" 的形式给出冷静的评估：收益结构并不相同——视觉编码器前向与超长视觉 prefill 在端到端延迟中占比可观，而投机解码只加速 decode；视觉条件的引入还改变了 draft 模型的校准性质。论文同时检验了 diffusion-based 并行起草在多模态场景的可行性边界。结论不是否定，而是提醒：多模态的投机解码需要跨模态改造 draft 设计，并配合视觉侧压缩才能兑现端到端收益。

5.6 Inference-time scaling 的效率代价

test-time compute 是 2025 年最耀眼的能力杠杆：更长的思维链换来数学与推理基准的大幅跳升（2503.18071 综述系统梳理了这条线）。但站在效率视角，它的账单触目惊心——长 CoT 把 decode 长度拉长一至两个数量级，KV cache 同步膨胀，而 decode 恰恰是最贵的阶段。多模态推理（先长篇看图描述、再逐步推理）使问题叠加。于是出现了一批" 效率化的推理扩展" 工作：budget forcing、早停准则、把长 CoT 蒸馏回短链。2607.28573 在本地 computer-use agent 场景实证了 inference scaling 的收益-成本曲线，指出存在一个甜蜜点，过长的推理链在端侧不仅慢而且收益饱和。" 智能/焦耳" 正在取代" 智能/参数" 成为新的评价维度——这或许是效率方向未来三年最重要的叙事转变。对 MLLM 还有一个独有变量：视觉输入分辨率本身就是 test-time compute 的一部分（更多切图 = 更多 token = 更贵也更准），于是" 看多久、想多久、花多少" 构成一个三维的资源分配问题，目前没有任何框架给出统一的答案。

6 技术路线对比与权衡

模型压缩三线对比：

维度	量化（GPTQ/AWQ）	剪枝（SparseGPT/LLM-Pruner）	蒸馏/小模型（TinyLLaVA 配方）
压缩对象	每参数的比特数	参数数量	架构规模本身
精度损失	4-bit 近无损，3-bit 可用	非结构化 50% 可用；结构化更伤	配方得当时小模型逼近大模型
硬件收益	显存与带宽（需定制 kernel）	非结构化几乎无实测加速；结构化真实加速	全面真实收益
额外成本	少量校准数据，小时级	剪后通常需恢复微调	完整训练流程，成本最高

维度	量化（GPTQ/AWQ）	剪枝（SparseGPT/LLM-Pruner）	蒸馏/小模型（TinyLLaVA 配方）
多模态适配	AWQ 已验证；视觉塔敏感层需保护	视觉塔与 LLM 可分别剪	端到端天然适配

解码加速与系统层对比：

技术	加速机制	精度	适用阶段	典型收益
投机解码（EAGLE 系列）	一次权重读取产出多 token	无损（分布一致）	decode	2.7–6.5×
PagedAttention（vLLM）	消除碎片、增大 batch	无损	全程（系统）	吞吐 2–4×
RadixAttention（SGLang）	共享前缀 KV 复用	无损	prefill（多请求/agent）	复杂工作流至 6.4×
KV 量化/驱逐	缩小 KV 字节数	近似无损至有损	decode 显存	上下文长度数倍扩展
视觉 token 压缩	缩短输入序列	任务依赖	prefill 与 KV	FLOPs -45% 至 14× token 压缩

文字分析。三条线不是竞争而是乘法关系：量化解决" 权重读多少字节"，投机解码解决" 读一次权重产出几个 token"，token 压缩与 KV 管理解决" 序列有多长、缓存有多大"，端侧小模型则把等式整体降一个量级。实际部署中它们几乎总是叠加使用——MiniCPM-V 上手机 = 8B 小模型 × int4 量化 × 高分辨率 token 预算控制。权衡的真正难点在交叉点：量化可能改变注意力分布从而影响 FastV 式剪枝的重要性判断；投机解码 draft 模型在量化目标模型上的校准会漂移；KV 驱逐与投机解码的树验证共享显存预算。2025 年的研究开始正视这些耦合（如 2604.12035 之于剪枝与校准），但系统的" 组合优化理论" 仍然缺位——这是工程现状，也是研究机会。

再给一张" 什么场景用什么招" 的速查表：

场景	首要瓶颈	首选技术组合
手机端单轮 VQA	显存容量、功耗	2–8B 小模型 + int4 量化（AWQ）+ 视觉 token 预算控制
云端高并发 API	吞吐、显存利用率	vLLM/SGLang + continuous batching + 前缀缓存 + EAGLE
长文档/视频理解	KV cache、prefill 长度	视觉 token 压缩 + KV 量化/驱逐 + chunked prefill
Agent 长轨迹	前缀冗余、KV 爆炸	RadixAttention 前缀复用 + 截图时间冗余压缩
深度推理（长 CoT）	decode 长度	投机解码 + budget forcing + 推理链蒸馏

最后一行的提醒：如果你只能记住本报告的一句话，请记住——效率优化是一个乘法游戏，任何单一技术都会被其他环节的瓶颈稀释，端到端收益永远等于各环节改善的乘积再乘以它们的耦合系数。

7 开放问题与未来方向

以下八个问题按" 理论—方法—系统" 的张力排列，前三个偏基础，中间三个偏方法，最后两个偏系统与评测。它们的共同特征是：都不需要从头训练大模型，但都要求对现有栈有解剖级的理解——这正是效率方向对学术界的友好之处。

1. 压缩的信息论下界。一张图" 值" 多少 token？视觉 token 压缩实验普遍显示可压掉一半甚至 14 倍而性能不降，但没有人给出任务条件下的下界刻画。压缩率-任务性能的相变曲线是否存在普适形式？
2. 组合压缩的相互作用理论。量化、剪枝、token 压缩、投机解码两两组合时误差如何传播？目前全部靠经验试凑，缺乏一个统一的敏感性分析框架。
3. 多模态 KV 的语义驱逐。现有 KV 驱逐策略（H2O/SnapKV）为文本设计，依据注意力统计；视觉 token 有空间结构（相邻 patch、切分网格），能否利用二维归纳偏置做更好的驱逐与压缩？ReVision（2605.11212）对连续截图利用时间冗余是第一步。
4. 端侧的" 有效参数" 架构。Gemma 3n 的 MatFormer + PLE 暗示一个方向：名义参数大、驻留参数小、按需物化。如何把这一思想与 MoE、量化统一成端侧架构的标准设计空间？
5. 推理扩展的成本感。test-time compute 需要自己的" 效率学"：给定延迟/能耗预算，推理链长度、草稿策略、视觉分辨率的最优联合分配是什么？
6. draft 模型的跨模态设计。视觉条件如何进入 draft？视觉 prefill 与文本 decode 的延迟占比随任务剧烈变化，静态的 draft 配置难以通吃。
7. 端侧多模态的能耗模型。延迟好测，能耗难评。手机 NPU、GPU、CPU 异构调度下，一次 VQA 的焦耳数如何建模与优化？现有工作（如 2609.25415 的无人机部署比较）刚开了头，缺乏公开的标准测量协议与基准。
8. 压缩的视觉编码器是否还能" 看见细节"。所有视觉 token 压缩都在拿 VQA 类基准验证，但 VQA 对细粒度空间信息的敏感度有限。压缩对 grounding、计数、OCR、图表理解的损伤是否被系统性低估？这关系到压缩技术能否进入文档智能与 GUI agent 等真正赚钱的场景。

8 研究机会点评（给入门研究者的建议）

算力门槛适中、问题清晰的子方向：

- 视觉 token 压缩仍是性价比最高的切入点。单卡即可复现 FastV/PruMerge，评测基准现成，问题定义干净（保性能前提下的压缩率），且 2025 年的" 可恢复压缩" 转向意味着旧方法的失败案例就是新方法的动机——从"FastV 在 OCR 上为什么掉点" 这种问题出发，一个学期足以做出有顶会相的工作。
- 小模型配方研究（TinyLLaVA Factory 路线）。2–4 张消费级 GPU 可做完整消融，社区缺的

是诚实的受控实验而非又一个大模型。

- 投机解码的多模态改造。EAGLE 已开源且被主流 serving 栈集成，在其上做视觉条件 draft、视觉 token 预算与接受率的联合分析，工程与学术价值兼备。

需要清醒认识的" 巨头游戏":

- 从头训练端侧旗舰（MiniCPM-V/Gemma 3n 级别）依赖海量私有数据与对齐工程，学术组无法在正面战场竞争，但可以在其开放的权重上做后训练、量化、部署侧研究。
- serving 系统核心（vLLM/SGLang 主线）迭代极快、工程壁垒高，个人贡献宜从插件、kernel、特定负载优化切入，而非另起炉灶。
- 极限低比特量化（2-bit 以下）的算法创新空间仍在，但需要扎实的优化理论与 kernel 功底，属于" 窄门深井" 型方向。

方法论提示：这个方向最适合的研究范式是" 诊断先行"。FastV 先画注意力曲线再设计方法，EAGLE-2 先发现接受率的上下文依赖，AWQ 先统计激活分布——最好的效率论文都是先给出一个可复现的、令人意外的系统观察，再让方法从这个观察里自然长出来。反过来，上来就堆模块的" 方法先行" 工作几乎没有幸存者。对入门者，这意味着你的第一个实验不应该是" 我想了个新剪枝策略"，而应该是" 我把某个模型的某个量可视化出来看看长什么样"。

一条普适建议：效率方向的论文评审越来越反感" 只报加速比不报任务保真度"，也越来越反感" 只在单基准报无损"。无论做哪个子方向，把效率-质量的完整帕累托曲线画出来、并诚实标注失败任务，是你与灌水工作拉开距离的最便宜方式。

9 术语表

- Prefill / Decode：推理两阶段；prefill 并行处理全部输入（计算受限），decode 逐 token 生成（访存受限）。
- KV cache：缓存历史 token 的 key/value 以避免重复注意力计算，显存占用随序列线性增长。
- 算术强度（Arithmetic Intensity）：每字节访存对应的浮点运算数，决定一段计算是计算受限还是访存受限。
- PTQ / QAT：训练后量化 / 量化感知训练；本报告讨论的 GPTQ/AWQ 均属 PTQ。
- Weight-only 量化（W4A16）：仅量化权重、激活保持 FP16；省显存与带宽但不省算力。
- 激活 outlier：激活中少量幅值极大的通道，是 LLM 量化的主要误差来源。
- Salient 权重：AWQ 概念，指位于大激活通道上、对量化误差最敏感的约 1% 权重。
- 结构化 / 非结构化剪枝：前者整块删除（通道、层）可直接加速；后者零散置零，需稀疏硬件才有实测收益。
- PagedAttention：vLLM 的 KV cache 管理算法，借鉴 OS 分页，block 化分配 + 引用计数共享。
- Continuous batching：逐 token 粒度调度请求进出 batch，消灭 padding 空转。
- RadixAttention：SGLang 的前缀复用机制，用基数树缓存并自动匹配共享前缀的 KV。

- Speculative decoding：小模型/自起草多 token、大模型并行验证的无损加速范式。
- 接受率（Acceptance rate）：draft token 被目标模型验证通过的比例，决定投机解码实际加速比。
- Tree attention：一次前向并行验证整棵候选 token 树的注意力掩码技术。
- Attention sink：注意力分数集中倾泻在序列首几个 token 的现象；StreamingLLM 据此设计流式 KV。
- 视觉 token 压缩：在进入或经过 LLM 时减少视觉 token 数量的技术总称（merge/prune/resample/pool）。
- Pixel shuffle：空间相邻特征拼入通道维的降采样，InternVL 系列用于视觉 token 减半。
- MatFormer：嵌套式 Transformer，单一权重可切出多个子模型尺寸，Gemma 3n 采用。
- Disaggregated serving：prefill 与 decode 拆分部署到不同硬件池的服务架构。
- Test-time compute：以增加推理时计算（长 CoT、多次采样）换取能力的范式，效率的对立面与新课题。

10 参考文献

[1] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Tri Dao et al., 2022, arXiv:2205.14135, https://arxiv.org/abs/2205.14135

[2] FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, Tri Dao, 2023, arXiv:2307.08691, https://arxiv.org/abs/2307.08691

[3] LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, Tim Dettmers et al., 2022, arXiv:2208.07339, https://arxiv.org/abs/2208.07339

[4] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, Elias Frantar et al., 2022, arXiv:2210.17323, https://arxiv.org/abs/2210.17323

[5] SmoothQuant: Accurate and Efficient Post-Training Quantization for LLMs, Guangxuan Xiao et al., 2022, arXiv:2211.10438, https://arxiv.org/abs/2211.10438

[6] AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration, Ji Lin et al., 2023, arXiv:2306.00978, https://arxiv.org/abs/2306.00978

[7] SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression, Tim Dettmers et al., 2023, arXiv:2306.03078, https://arxiv.org/abs/2306.03078

[8] QLoRA: Efficient Finetuning of Quantized LLMs, Tim Dettmers et al., 2023, arXiv:2305.14314, https://arxiv.org/abs/2305.14314

[9] SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot, Elias Frantar, Dan Alistarh, 2023, arXiv:2301.00774, https://arxiv.org/abs/2301.00774

[10] Wanda: A Simple and Effective Pruning Approach for Large Language Models, Mingjie Sun et al., 2023, arXiv:2306.11695, https://arxiv.org/abs/2306.11695

[11] LLM-Pruner: On the Structural Pruning of Large Language Models, Xinyin Ma et al., 2023, arXiv:2305.11627, https://arxiv.org/abs/2305.11627

[12] SliceGPT: Compress LLMs by Deleting Rows and Columns, Saleh Ashkboos et al., 2024, arXiv:2401.15024, https://arxiv.org/abs/2401.15024

[13] Accelerating LLM Decoding with Speculative Sampling, Charlie Chen et al. (DeepMind), 2023, arXiv:2302.01318,

https://arxiv.org/abs/2302.01318

[14] Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads, Tianle Cai et al., 2024, arXiv:2401.10774, https://arxiv.org/abs/2401.10774

[15] EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty, Yuhui Li et al., 2024, arXiv:2401.15077, https://arxiv.org/abs/2401.15077

[16] EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees, Yuhui Li et al., 2024, arXiv:2406.16858, https://arxiv.org/abs/2406.16858

[17] EAGLE-3: Scaling up Inference Acceleration via Training-Time Test, Yuhui Li et al., 2025, arXiv:2503.01840, https://arxiv.org/abs/2503.01840

[18] Efficient Memory Management for LLM Serving with PagedAttention (vLLM), Woosuk Kwon et al., 2023, arXiv:2309.06180, https://arxiv.org/abs/2309.06180

[19] SGLang: Efficient Execution of Structured Language Model Programs, Lianmin Zheng et al., 2023, arXiv:2312.07104, https://arxiv.org/abs/2312.07104

[20] H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs, Zhenyu Zhang et al., 2023, arXiv:2306.14048, https://arxiv.org/abs/2306.14048

[21] StreamingLLM: Efficient Streaming Language Models with Attention Sinks, Guangxuan Xiao et al., 2023, arXiv:2309.17453, https://arxiv.org/abs/2309.17453

[22] SnapKV: LLM Knows What You are Looking for Before Generation, Yuhong Li et al., 2024, arXiv:2404.14469, https://arxiv.org/abs/2404.14469

[23] KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization, Coleman Hooper et al., 2024, arXiv:2401.18079, https://arxiv.org/abs/2401.18079

[24] Token Merging: Your ViT but Faster (ToMe), Daniel Bolya et al., 2022, arXiv:2210.09461, https://arxiv.org/abs/2210.09461

[25] FastV: An Image is Worth 1/2 Tokens After Layer 2, Liang Chen et al., 2024, arXiv:2403.06764, https://arxiv.org/abs/2403.06764

[26] LLaVA-PruMerge: Adaptive Token Reduction for Efficient Large Multimodal Models, Yuzhang Shang et al., 2024, arXiv:2403.15388, https://arxiv.org/abs/2403.15388

[27] LLaMA-VID: An Image is Worth 2 Tokens in Large Language Models, Yanwei Li et al., 2023, arXiv:2311.17043, https://arxiv.org/abs/2311.17043

[28] TinyLLaVA Factory: A Framework of Small-scale Large Multimodal Models, Jiajun Zhou et al., 2024, arXiv:2405.11788, https://arxiv.org/abs/2405.11788

[29] MoE-LLaVA: Mixture of Experts for Large Vision-Language Models, Bin Lin et al., 2024, arXiv:2401.15947, https://arxiv.org/abs/2401.15947

[30] Align-KD: Distilling Cross-Modal Alignment Knowledge for Mobile Vision-Language Models, 2024, arXiv:2412.01282, https://arxiv.org/abs/2412.01282

[31] MiniCPM-V: A GPT-4V Level MLLM on Your Phone, Yuan Yao et al. (OpenBMB), 2024, arXiv:2408.01800, https://arxiv.org/abs/2408.01800

[32] MobileVLM: A Fast, Strong and Open Vision Language Assistant for Mobile Devices, Xiangxiang Chu et al., 2023, arXiv:2312.16886, https://arxiv.org/abs/2312.16886

[33] MobileVLM V2: Faster and Stronger Baseline for Vision Language Model, Xiangxiang Chu et al., 2024, arXiv:2402.03766, https://arxiv.org/abs/2402.03766

[34] Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution, Peng

Wang et al., 2024, arXiv:2409.12191, https://arxiv.org/abs/2409.12191

[35] InternVL 2.5: Expanding Performance Boundaries of Open-Source Multimodal Models, Zhe Chen et al., 2024, arXiv:2412.05271, https://arxiv.org/abs/2412.05271

[36] DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding, Zhiyu Wu et al., 2024, arXiv:2412.10302, https://arxiv.org/abs/2412.10302

[37] LLaVA-OneVision: Easy Visual Task Transfer, Bo Li et al., 2024, arXiv:2408.03326, https://arxiv.org/abs/2408.03326

[38] Gemma 3 Technical Report, Gemma Team (Google), 2025, arXiv:2503.19786, https://arxiv.org/abs/2503.19786

[39] Gemma 3n: Mobile-First Multimodal Model with MatFormer and Per-Layer Embeddings, Google, 2025, [官方发布/模型卡，无 arXiv]

[40] SmolVLM, Hugging Face, 2024–2025, [官方发布/模型卡，无 arXiv]

[41] moondream, vikhyat/moondream, 2024–, [开源发布/模型卡，无 arXiv]

[42] Efficient Multimodal Large Language Models: A Survey, Yizhang Jin et al., 2024, arXiv:2405.10739, https://arxiv.org/abs/2405.10739

[43] A Survey on Large Language Model Acceleration based on KV Cache Management, Haoyu Luo et al., 2024, arXiv:2412.19442, https://arxiv.org/abs/2412.19442

[44] Unlocking Efficiency in LLM Inference: A Comprehensive Survey of Speculative Decoding, Heming Xia et al., 2024, arXiv:2401.07851, https://arxiv.org/abs/2401.07851

[45] A Survey of Token Compression for Efficient Multimodal Large Language Models, 2025, arXiv:2507.20198, https://arxiv.org/abs/2507.20198

[46] Mind with Eyes: from Language Reasoning to Multimodal Reasoning, 2025, arXiv:2503.18071, https://arxiv.org/abs/2503.18071

[47] Ask in Any Modality: A Comprehensive Survey on Multimodal Retrieval-Augmented Generation, 2025, arXiv:2502.08826, https://arxiv.org/abs/2502.08826